Pesquisa Hook

Data de Início	6 de abril de 2026
Status	Concluído

Letícia Roberta Oliveira Souto

— O que é

São funções especiais no React Native que permitem gerenciar o estado e recursos de ciclo de vida em componentes funcionais, sem usar classes.

- Não é necessário mais escrever componentes longos e complexos.
- Lógica mais clara.
- Possível criar ganchos personalizados para reutilizar a lógica em vários componentes.
- Evitem renderizações desnecessárias, tornando o aplicativo mais rápido.

— Hook `useState`

Este gancho permite adicionar estado ao seu componente funcional. É utilizado para lidar cpm estados de interface de usuário simples. como botões de alternância, contadores e valores de entrada.

É possível utilizar useState mais de uma vez em um único componente. A sintaxe de desestruturação de arraysnos permite atribuir diferentes nomes para as variáveis de state que declaramos chamando useState. Esses nomes não fazem parte da API useState. Em vez disso, React presume que se você chamar useState muitas vezes, você faz isso na mesma ordem a cada renderização.

Exemplo:

```
import React, { useState } from 'react';

function Example() {
  // Declara uma nova variável de state, que chamaremos de
"count"
```

```
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Você clicou {count} vezes</p>
      <button onClick={() => setCount(count + 1)}>
        Clique aqui
      </button>
    </div>
  );
}
```

— Hook `useEffect`

É um gancho usado quando é necessário executar efeitos colaterais através de um componente funcional, como obter/buscar dados de uma API, inscrição em eventos, gerenciar temporizadores e atualizar dados.

Exemplo:

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  // Similar a componentDidMount e componentDidUpdate:
  useEffect(() => {
    // Atualiza o título do documento utilizando a API do n
avegador
    document.title = `Você clicou ${count} vezes`;
  });

  return (
    <div>
      <p>Você clicou {count} vezes</p>
      <button onClick={() => setCount(count + 1)}>
        Clique aqui
```

```
      </button>
    </div>
  );
}
```

Quando você chama `useEffect`, você está dizendo ao React para executar a sua função de “efeito” após liberar as mudanças para o DOM. Efeitos são declarados dentro do componente, para que eles tenham acesso as suas props e state. Por padrão, React executa os efeitos após cada renderização, incluindo a primeira renderização.

— Hook `useRef`

É um Hook do React que cria um objeto mutável persistete entre renderizações, permitindo armazenar valores (como IDs de temporizadores ou referências a omponentes) sem engatilhar nova renderização da interface. É ideal para acessar diretamente componentes nativos.

Exemplo:

```
import React, { useRef } from 'react';
import { TextInput, Button, View } from 'react-native';

const App = () => {
  const inputRef = useRef(null);

  const focucarInput = () => {
    // Acessando o método focus() do componente TextInput
    inputRef.current.focus();
  };

  return (
    <View>
      <TextInput ref={inputRef} placeholder="Clique no botã
o para focar" />
      <Button title="Focar" onPress={focucarInput} />
    </View>
  );
};
```

— Array no `useEffect`

O array de dependências no `useEffect` determina quando o efeito deve ser reexecutado. O comportamento muda drasticamente dependendo se ele está vazio, preenchido ou ausente.

Quando está vazio o efeito roda apenas uma vez, logo após o componente ser montado (renderizado pela primeira vez). Equanto quando ele está ausente o efeito roda após cada renderização (montagem + atualização), toda vez que o componente atualizar ser estado ou receber novas props, o efeito será executado novamente.

— Diferença entre `useRef` e `useState`

A principal diferença entá na renderização do componente e na atualização do usuário, o `useState` tem a finalidade de armazenar dados que afetam a renderização da UI e suas atualização são assíncronas. Já o `useRef` tem a finalidade de armazenar valores mutáveis que persistem entre renderizações, mas que não precisar ser exibidos na tela imediatamente, além de sua atualização ser síncrona, esta é especialmente útil em cenários onde você precisa "lembrar" um valor, mas não deseja que a tela pisque ou atualize toda vez que esse valor mudar. Ou seja, se o valor for exibido na tela, é utilizado o `useState`, mas se for apenas para lógica interna é utilizado o `useRef.`

— Regras dos Hooks

- Apenas chame Hooks no nível mais alto. Não chame Hooks dentro de loops, condições ou funções aninhadas.
- Apenas chame Hooks de componentes funcionais. Não came Hooks de funções JavaScript comuns → Há apenas um outro lugar válido para se chamar hooks (dentro dos seus próprios Hooks customizados)

O React não sabe quais hooks mudaram, apenas que eles foram chamados na mesma sequência. Se um hook está dentro de um `if` e esse `if` for falso, a ordem dos hooks muda, o que faz com que o React associe dados de estado a ganchos incorrretos.

— Fontes

- https://medium.com/@CodeCraftMobile/react-native-hooks-a-beginners-guide-a5ef44563f8e
- https://pt-br.legacy.reactjs.org/docs/hooks-overview.html